

Pillar 축 소실 실험: Pedestrian·Cyclist 결과와 Z축 정보 손실 사례

PointPillars의 필라 형성 평면을 XY/XZ/YZ로 바꿔가며 학습한 축-소실 (ablation) 실험의 후속 보고 — 이번엔 Pedestrian·Cyclist 클래스로 확장

Dataset KITTI object detection (train 3,712 / val 3,769)
Model PointPillars, xyres_16, batch 2, 296,960 step (160 epoch) Classes Cyclist + Pedestrian (단일 모델)
Repo DILAB-pointpillars @ 9ed03a3

1. 개요 2. 실험 설계 3. 결과 4. 왜 XZ/YZ는 무너지는가 5. Z축 정보 손실 사례 6. 결론과 다음 단계 부록 · 재현 정보

1 개요

기존에는 Car 클래스에 대해서만 필라(pillar)를 세 가지 평면 — 표준 XY(Z 소실), XZ(Y 소실), YZ(X 소실) — 으로 각각 형성해 학습·비교했다. 이번 실험은 동일한 축-소실 설계를 **Pedestrian·Cyclist** 클래스로 확장해, 작고 얇은 물체에서도 같은 경향이 나타나는지 확인한 것이다. 아울러 지도교수님이 요청하신 **"Z축(높이) 정보가 실제로 손실되는 구체적 상황"**에 대한 근거를 코드 분석과 실제 KITTI 장면 사례 분석 두 갈래로 제시한다 (5장).

2 실험 설계

세 실험은 second/core/view_transform.py의 view 인자(xy/xz/yz)만 다르고, 나머지 — 데이터 분할, 앵커/타겟 매칭, 학습 스케줄, 손실 함수, 평가 프로토콜 — 는 완전히 동일하다. 즉 "필라를 어느 두 축 위에 형성하고 나머지 한 축을 하나의 bin(bin)으로 접을지"만 통제된 단일 변인 실험이다.

세 실험(view)의 정의

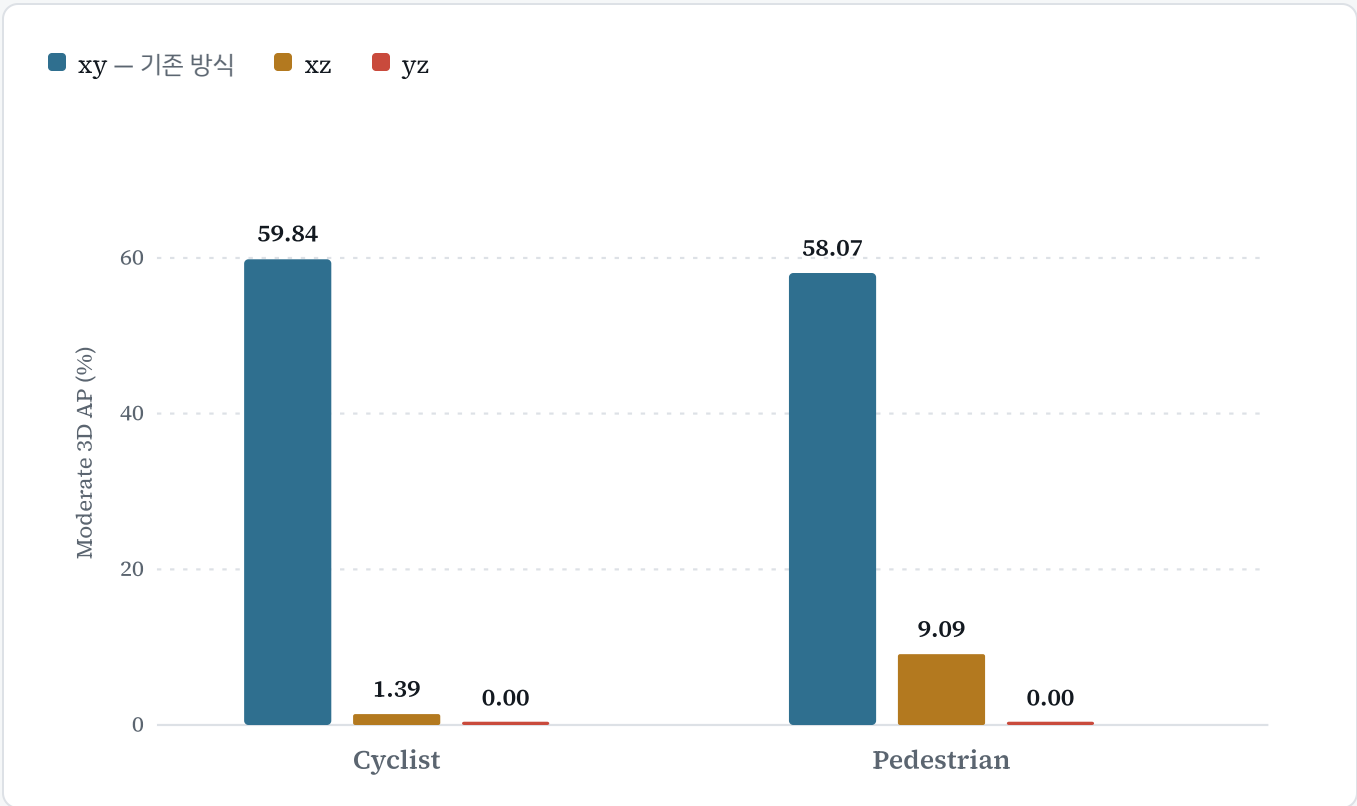
VIEW	필라 평면(공간 축)	소실(단일 bin으로 접히는) 축
■ xy — 기존 PointPillars	X, Y (지면 평면 · BEV)	Z (높이)
■ xz	X, Z	Y

VIEW	필라 평면(공간 축)	소실(단일 빈으로 접히는) 축
yz	Y, Z	X

학습 스케줄은 세 경우 모두 동일: 296,960 step(1,856 step/epoch × 160 epoch), batch 2, Adam, decay 15 epoch 주기. 평가는 KITTI 공식 프로토콜(easy / moderate / hard, Cyclist·Pedestrian 모두 IoU 0.5)을 그대로 사용했다.

2 결과

세 축 모두 **Moderate** 난이도 3D AP를 기준으로 비교하면 다음과 같다 (수치가 클수록 좋음):



KITTI 공식 평가 (IoU 0.5) · easy / moderate / hard, 클래스별 3D·BEV AP (%)

VIEW	CLASS	3D AP	BEV AP
xy	Cyclist	81.46 / 59.84 / 56.23	82.92 / 61.79 / 58.59
xy	Pedestrian	62.23 / 58.07 / 52.77	71.06 / 66.44 / 60.62
xz	Cyclist	1.66 / 1.39 / 1.38	2.10 / 1.73 / 1.62
xz	Pedestrian	9.45 / 9.09 / 9.09	9.96 / 9.68 / 9.64
yz	Cyclist	0.00 / 0.00 / 0.00	0.00 / 0.00 / 0.00



yz

Pedestrian

0.00 / 0.00 / 0.00

0.00 / 0.00 / 0.00

보조 지표(COCO식 AP@IoU 0.25-0.70 평균, 3D)도 동일한 경향을 보인다 — xy: Cyclist 52.5 / Pedestrian 52.1(moderate)
대 xz: 1.5 / 5.0 대 yz: 0.00 / 0.00.

이전 Car 실험(별도 환경에서 완료, 본 리포트 범위 밖)과 마찬가지로 **xy(기준) >> xz > yz** 순서가 그대로 재현되었다. Pedestrian·Cyclist는 Car보다 작고 얇은 물체인데도 축을 바꿨을 때의 붕괴 양상은 오히려 더 극단적이다(yz는 사실상 완전 학습 실패).

3 왜 XZ·YZ는 이렇게 무너지는가

직관적으로는 "Z(높이)를 명시적으로 살려두면 더 잘 될 것" 같지만 정반대로 나타난다. 학습 로그에서 확인한 두 가지 요인이 크다.

1. **앵커 프루닝이 꺼진다.** second/builder/dataset_builder.py는 view != 'xy'일 때 anchor_area_threshold = -1로 강제 설정한다 — BEV 점유 기반으로 빈 앵커를 걸러내는 최적화(anchors_mask)가 XY 그리드 전용이라 다른 평면에서는 끝 수밖에 없기 때문이다. 그 결과 유효 앵커 수가 급증한다: 학습 로그 기준 xy는 프레임당 앵커 수가 대략 20,000 ~ 91,000개로 변동하는 반면, xz/yz는 프레임에 관계없이 항상 293,632개로 고정된다. 양성 앵커(num_pos)는 40~120개 수준으로 비슷한데 음성 앵커만 3배 넘게 불어나니, 클래스 불균형이 그만큼 심해진다.
2. **애초에 XY 평면이 정보량 자체가 더 많다.** 차량 탑재 LiDAR는 방위각(azimuth) 방향으로 촘촘히 스캔하고 고도(elevation) 방향은 채널(스캔라인) 수만큼만 성긴 해상도를 갖는다. 그래서 같은 필라 그리드 해상도(0.16m)를 Z에 적용해도 실제로 채워지는 정보는 X·Y보다 훨씬 성기고, 보행자·자전거처럼 애초에 높이 방향 변화가 작은 물체는 더더욱 그렇다.

즉 이 실험은 "Z 정보가 원래 쓸모없다"를 보여주는 게 아니라, ①**앵커 불균형 심화** + ②**Z축 자체의 낮은 유효 해상도**가 겹친 결과로 읽어야 한다. 두 요인이 섞여 있어 xz/yz의 하락 폭 자체를 "Z정보의 순수한 가치"로 곧이곧대로 해석하기는 어렵다 — 다만 xy가 압도적으로 우세하다는 **방향**은 이 정도 격차에서는 충분히 신뢰할 만하다.

4 Z축 정보 손실 사례 — 근거 있는 답변

지도교수님 질문: "Z축이 손실되는 특정 상황을 찾아보라"에 대해, (A) 코드 상에서 Z가 정확히 어떻게 사라지는지와 (B) 그로 인해 실제로 놓친 KITTI 사례를 근거로 제시한다.

4.1 코드 근거: 필라 하나 = Z축 전체를 한 번에 뭉겐 값

표준 xy 파이프라인에서 Z가 사라지는 지점은 정확히 두 곳이다.

- **필라 격자 자체가 Z를 1개 구간으로 정의한다.** second/core/view_transform.py의 make_pillar_voxel_config는 소실 축(xy에서는 Z)의 voxel 크기를 point_cloud_range 전체 폭 + 1.0으로 설정해, 그 축은 그리드 셀이 정확히 1개가 되도록 만든다 — 어떤 높이의 점이든 같은 (x, y) 위치라면 전부 같은 필라에 들어간다.
- **필라 안에서는 최고값 하나로 요약된다.** second/pytorch/models/pointpillars.py의 PFNLayer.forward는 `x_max = torch.max(x, dim=1, keepdim=True)[0]` — 필라 하나에 속한 모든 점(=그 (x,y) 위치의 전체 높이 범위)에 대해 채널별 최대값 하나만 남기고 나머지는 버린다.

이 둘을 합치면: **같은 지면 위치(x, y)에 서로 다른 높이의 구조물이 겹쳐 있으면, RPN에 도달하기도 전에 하나의 특징 벡터로 뭉개진다.** 즉 "보행자가 자기 키 높이만큼의 점을 갖고 있다"는 사실과 "그 보행자 바로 위/옆에 가로수·표지판·건물 처마가 있다"는 사실이 구분되지 않는다 — 이것이 Z가 손실되는 정확한 메커니즘이다.

4.2 실증 분석: 실제로 놓친 사례에서 확인

위 메커니즘이 실제 탐지 실패로 이어지는지 확인하기 위해, xy 모델의 검증셋(val, 3,769장) 예측 결과 (result.pkl)를 정답과 클래스별 BEV IoU(0.5, score>0.3)로 매칭해 **놓친 물체(FN)**와 **맞춘 물체(TP)**를 나누고, 각 물체의 지면 발자국(bounding box를 사방 +0.25m 확장) 안에서 "그 물체 자신의 높이 범위보다 위쪽에 있는 라이다 점 개수"(above-clutter)를 비교했다.

PEDESTRIAN 매칭

1,448 / 724

TP / FN · recall
66.7%

CYCLIST 매칭

416 / 184

TP / FN · recall
69.3%

PED. ABOVE-CLUTTER

0.22 → 1.00

TP 평균 → FN 평균 (점 개수)

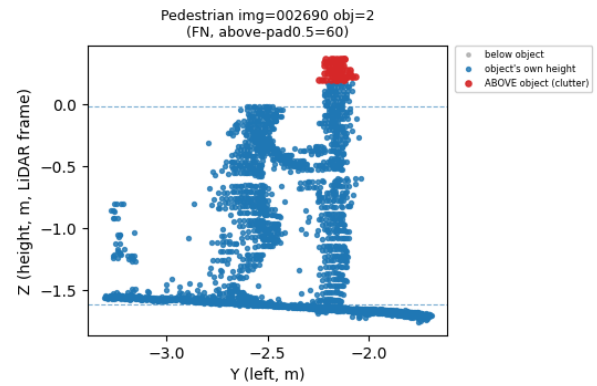
CYC. ABOVE-CLUTTER

0.08 → 1.12

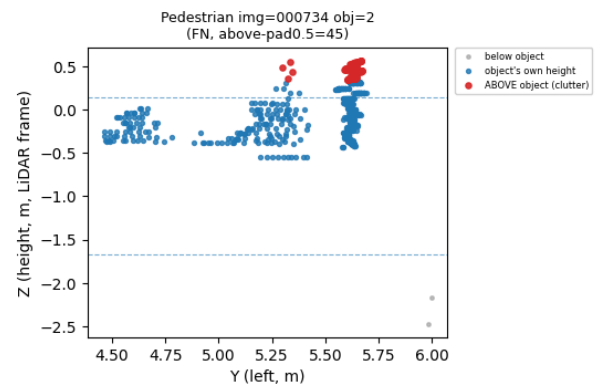
TP 평균 → FN 평균 (점 개수)

FN 쪽이 TP보다 발자국 위쪽 점(above-clutter)이 확연히 많다 (Mann-Whitney U, one-sided: Pedestrian $p=1.1 \times 10^{-12}$, Cyclist $p=1.6 \times 10^{-26}$ — 둘 다 강한 통계적 유의성). FN 물체는 자기 몸통 점 개수 자체도 TP보다 적은 경향(원거리·부분 가림)이 있어, "자기 점 대비 위쪽 점 비율"로 다시 보면 격차는 더 벌어진다 — Pedestrian 약 17배(TP 0.0011 → FN 0.0190), Cyclist 약 12배(TP 0.0012 → FN 0.0148), 둘 다 $p < 10^{-12}$.

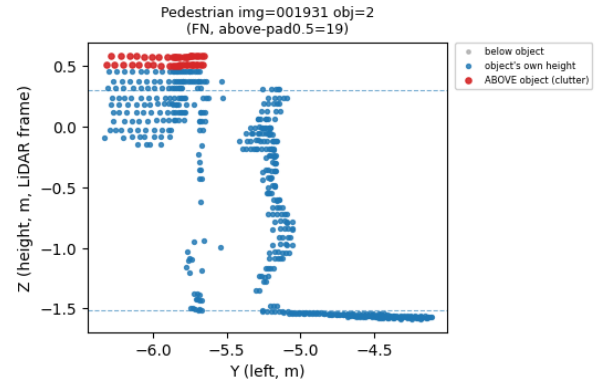
아래는 above-clutter가 가장 뚜렷했던 실제 FN 사례 4건이다. 왼쪽은 카메라 영상(정답 박스 주변 크롭), 오른쪽은 그 물체의 지면 발자국 안 라이다 점을 높이(Z) 기준으로 그린 산점도 — 파란 점이 물체 자신의 높이 범위, 빨간 점이 그 위쪽에서 잡힌 "다른 무언가"다.



신호등 기둥에 바짝 붙어 선 보행자 2인. 머리 위로 가로수 캐노피가 짙게 드리워 있다.



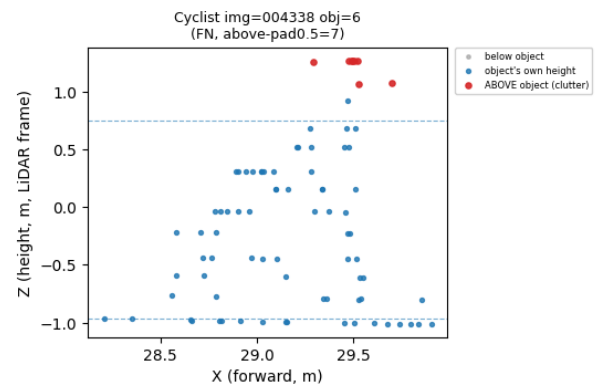
건물 벽 모퉁이 옆, 표지판과 주차 차량 사이에 선 보행자. (동일 인물이 인접 프레임 000401·007260에도 거의 같은 모습으로 반복 등장 — 독립 사례 아님, 4.2절 한계 참고)



건물 처마/차양 아래, 표지판 기둥 옆에 서 있는 보행자.

img 004338 · Cyclist FN · above=7

KITTI val



교통 표지판/신호등 기둥 옆을 지나가는 자전거 탑승자.

방법론 한계: (1) 매칭은 공식 KITTI 프로토콜이 아닌 근사적 BEV-IoU 방식이다. (2) above-clutter는 정확한 박스 발자국 이 아니라 +0.25m 확장 발자국 기준 — 정확한 발자국만 보면 신호가 훨씬 약해진다(사례가 드물어서다). "가까이 서 있다"는 프레임에 맞춰 확장 발자국을 주 지표로 채택했다. (3) KITTI는 나무·기둥·벽 같은 정적 구조물에 라벨을 달지 않으므로, 위 쪽 점이 "정확히 무엇"인지는 영상으로 짐작할 뿐 확정할 수 없다. (4) 사례 2(img 000734)와 동일한 정적 물체가 인접 프레임 (000401, 007260)에도 거의 그대로 나타나 — 통계에는 포함했지만 서로 독립적인 3개 사례는 아니다.

5 결론과 다음 단계

결론. Pedestrian·Cyclist에서도 Car와 동일하게 표준 XY 필라(BEV)가 XZ·YZ를 압도적으로 앞선다. 동시에, 그 XY 방식이 이론과 실측 양쪽에서 확인되는 진짜 약점 — *같은 지면 위치에서 높이가 다른 물체/구조물이 겹치면 하나의 특징으로 뭉개진다*는 것도 분명하다. 이번에 찾은 4건(+동일 물체 반복 2건)은 전부 "구조물 바로 옆/아래 서 있는 보행자·자전거" 패턴이었다.

다음 단계 제안. 지난 논의에서 나온 것처럼, 이미 코드에 XY/XZ/YZ 세 view 모두 동일한 canonical 그 리드로 리사이즈해 RPN에 넣는 구조(second/pytorch/models/voxelnet.py의 `_canonical_hw` 보간)가 있어 — 세 인코더를 따로 학습하는 대신 **채널 방향으로 합쳐(concat) 하나의 모델로 학습**하는 실험이 자연스러운 후속 실험이다. 다만 yz는 이번 결과처럼 거의 신호가 없어 fusion에 잡음만 보탬 수 있으니, 우선 **xy+xz 2-branch**로 가볍게 시도해보는 쪽을 권장한다.

부록 · 재현 정보

CONFIG

second/configs/pointpillars/ped_cycle/xyres_16.proto

CHECKPOINTS

second/model_dir_pedcycle_{xy,xz,yz}/ (voxelnet-296960.tckpt)

학습 로그

second/logs/pedcycle_{xy,xz,yz}.log

검증 예측

second/model_dir_pedcycle_xy/results/step_296960/result.pkl

실행 커맨드

```
PYTHONPATH=. CUDA_VISIBLE_DEVICES=<n> .venv/bin/python -m second.pytorch.train train -  
-config_path=<config> --model_dir=<dir> --view=<xy|xz|yz>
```